MySQL SQL Interview Practice

25 questions (13 medium, 12 hard) in MySQL 8.0+. Q1 to Q7 are from your assessment; Q8 to Q25 are new practice questions.

Each question has: problem, schema, sample input, expected output with ORDER BY, explanation, solution. Expected outputs were produced by running the solution logic on the sample data in SQLite (window functions, CTEs, recursion). Syntax was written for the dialect shown but was not executed on a live MySQL or BigQuery server, so confirm in your own console.

Contents

#	Title	Level	Topic
Q1	Pizza order final status	Medium	Conditional aggregation, CASE
Q2	Team player selection (running average)	Medium	Window running AVG, join on first letter
Q3	Highest-revenue customer (buy/sell rules)	Medium	SUM(CASE), ties with RANK
Q4	Top 5 cross-selling category pairs	Medium	Self-join, DISTINCT, pair counting
Q5	Top reps per region, ties included	Hard	DENSE_RANK, multi-CTE, 2026 filter
Q6	Top 2 earners per department, ties broken	Hard	ROW_NUMBER, LEAD before filter
Q7	Longest login streak per user	Hard	Gaps and islands, recursive CTE
Q8	Second highest distinct salary per department	Medium	DENSE_RANK, NULL when absent
Q9	Duplicate emails: which ids to delete	Medium	ROW_NUMBER, named WINDOW, FIRST_VALUE
Q10	Month-over-month revenue growth	Medium	LAG, DATE_FORMAT
Q11	Customers active in every one of the last 3 months	Medium	COUNT DISTINCT months, HAVING
Q12	Employees earning more than their manager	Medium	Self-join
Q13	Pivot: monthly sales by category	Medium	Conditional aggregation pivot
Q14	Median salary per department	Hard	ROW_NUMBER + COUNT window, even/odd handling
Q15	Products never sold, and dead categories	Medium	Anti-join, NOT EXISTS
Q16	First day cumulative sales reach 1000	Hard	Running SUM, first row after threshold
Q17	Sessionization with a 30-minute gap	Hard	LAG + running SUM of flags
Q18	Monthly cohort retention	Hard	Cohorts, month offsets, conditional distinct counts
Q19	Overlapping room bookings	Hard	Interval overlap self-join
Q20	Merge overlapping date ranges	Hard	Running MAX frame, island flag
Q21	All reports under a manager (recursive CTE)	Hard	WITH RECURSIVE, path building
Q22	People you may know (mutual friends)	Hard	Self-joins, NOT EXISTS, ranking with tiebreak
Q23	Rolling 7-calendar-day average with missing days	Hard	Range window vs correlated subquery
Q24	Top-selling product per category with share	Medium	RANK with ties, SUM OVER share
Q25	Users who flipped status at least twice	Medium	LAG, counting changes

Q1 Pizza order final status [Medium]

A pizza company stores each pizza as a separate row in `Customer_Order`. Status is CREATED, SUBMITTED or DELIVERED. Final_Status per customer, checked in order: (1) all orders DELIVERED -> COMPLETED; (2) some DELIVERED and some not -> IN PROGRESS; (3) at least one SUBMITTED and none DELIVERED -> AWAITING PROGRESS; (4) otherwise AWAITING SUBMISSION. Return customer_name and Final_Status.

Schema

`Customer_Order` (customer_name varchar(50), order_id varchar(10), status varchar(50))

Sample input

`Customer_Order`

customer_name	order_id	status
Alice	O1	DELIVERED
Alice	O2	DELIVERED
Bob	O3	DELIVERED
Bob	O4	SUBMITTED
Carol	O5	SUBMITTED
Carol	O6	CREATED
Dave	O7	CREATED
Dave	O8	CREATED
Eve	O9	CREATED
Eve	O10	DELIVERED

Expected output (ORDER BY customer_name)

customer_name	Final_Status
Alice	COMPLETED
Bob	IN PROGRESS
Carol	AWAITING PROGRESS
Dave	AWAITING SUBMISSION
Eve	IN PROGRESS

Explanation

- Alice: all delivered -> COMPLETED. Bob: 1 delivered + 1 not -> IN PROGRESS.
- Carol: no delivered, one SUBMITTED -> AWAITING PROGRESS. Dave: all CREATED -> AWAITING SUBMISSION.
- Eve: CREATED counts as 'not delivered', so CREATED + DELIVERED -> IN PROGRESS.
- CASE stops at the first true branch, so rule order replaces extra 'none delivered' checks. COUNT(CASE ... END) skips NULLs.

MySQL 8.0+

```
with base as (
  select customer_name,
         count(distinct case when status = 'SUBMITTED' then order_id end) as submitted,
         count(distinct case when status = 'DELIVERED' then order_id end) as delivered,
         count(distinct order_id)                                         as total_orders
  from Customer_Order
  group by customer_name
)
select customer_name,
       case when delivered = total_orders then 'COMPLETED'
            when delivered > 0            then 'IN PROGRESS'   -- rule 1 already caught all-delivered
            when submitted > 0            then 'AWAITING PROGRESS'
            else 'AWAITING SUBMISSION' end as Final_Status
from base
order by customer_name;
```

Q2 Team player selection (running average) [Medium]

A player is eligible for a team if the first letters of team and player names match. Each team has avg_age, the maximum allowed average age. Add eligible players in ascending order of age while the running average of the selected players stays <= avg_age. Return tname and number of players selected for each team that has players.

Schema

`teams` (tname varchar(15), avg_age int)
`players` (pid int, pname varchar(20), age int)

Sample input

`teams`

tname	avg_age
Alpha	20
Bravo	25
Delta	30
Echo	22

`players`

pid	pname	age
1	Adam	18
2	Alex	20
3	Anna	25
4	Ben	22
5	Bob	30
6	Bill	26
7	Dan	28
8	Dave	29
9	Zoe	40

Expected output (ORDER BY tname)

tname	num_players
Alpha	2
Bravo	2
Delta	2

Explanation

- Alpha: running averages 18, 19, 21; 21 > 20 so Anna is dropped -> 2. Bravo: 22, 24, 26 -> 2. Delta: 28, 28.5 -> 2.
- Echo has no player starting with E (inner join drops it). Zoe matches no team.
- Ascending ages mean the running average never decreases, so counting rows that pass is the same as stopping at the first failure.
- pid in ORDER BY makes equal ages deterministic. ROWS (not RANGE) treats tied ages as separate rows.

MySQL 8.0+

```
with e as (
  select t.tname, t.avg_age,
         avg(p.age) over (partition by t.tname order by p.age, p.pid
                          rows between unbounded preceding and current row) as run_avg
  from teams t
  join players p on upper(left(t.tname, 1)) = upper(left(p.pname, 1))
)
select tname, count(*) as num_players
from e
where run_avg <= avg_age
group by tname
order by tname;
```

Q3 Highest-revenue customer (buy/sell rules) [Medium]

Each transaction row has customer_id, type (BUY/SELL), amount, status. Revenue rules: COMPLETED BUY adds 100% of amount; COMPLETED SELL adds 10%; PENDING is ignored; CANCELLED (BUY or SELL) is void and deducts 1% of amount. Return the customer(s) with the highest total revenue (all ties) with their revenue.

Schema

`transactions` (customer_id int, type varchar(10), amount decimal(10,2), status varchar(15))

Sample input

`transactions`

customer_id	type	amount	status
1	BUY	1000	COMPLETED
1	SELL	500	COMPLETED
1	BUY	300	PENDING
2	BUY	800	COMPLETED
2	SELL	2000	COMPLETED
2	BUY	500	CANCELLED
3	SELL	5000	COMPLETED
3	BUY	700	COMPLETED
3	SELL	1000	CANCELLED
4	BUY	1200	PENDING
4	SELL	400	CANCELLED

Expected output (ORDER BY customer_id)

customer_id	revenue
3	1190.00

Explanation

- Customer 1 = 1000 + 50 = 1050. Customer 2 = 800 + 200 - 5 = 995. Customer 3 = 500 + 700 - 10 = 1190. Customer 4 = -4.
- CANCELLED branch ignores type because the 1% deduction applies to both. `else 0` covers PENDING.
- RANK gives every tied customer rank 1. Use ROW_NUMBER if exactly one row is wanted.

MySQL 8.0+

```
with r as (
  select customer_id,
         sum(case when status = 'COMPLETED' and type = 'BUY'  then amount
                  when status = 'COMPLETED' and type = 'SELL' then 0.10 * amount
                  when status = 'CANCELLED'                   then -0.01 * amount
                  else 0 end) as revenue
  from transactions
  group by customer_id
),
ranked as (select customer_id, revenue, rank() over (order by revenue desc) as rk from r)
select customer_id, revenue from ranked where rk = 1 order by customer_id;
```

Alternative without window functions

```
-- same CTE r as above, then:
select customer_id, revenue from r
where revenue = (select max(revenue) from r)
order by customer_id;
```

Note: SUM(CASE ...) without ELSE returns NULL for customers with no matching row; adding or subtracting NULL gives NULL. Always give ELSE 0.

Q4 Top 5 cross-selling category pairs [Medium]

Find the 5 pairs of product categories most commonly bought together (same order_id). A pair counts once per order and (A,B) equals (B,A). Return category_pair as 'cat1-cat2' (alphabetical) and times_bought_together.

Schema

`orders` (order_id int, product_id varchar(5), quantity int, price decimal(10,2), order_date date)
`product_dimensions` (product_id varchar(5), category_id int, category_name varchar(30))

Sample input

`orders`

order_id	product_id	quantity	price	order_date
1	P1	1	500	2026-10-01
1	P2	2	100	2026-10-01
2	P1	1	500	2026-10-02
2	P2	1	100	2026-10-02
2	P3	1	50	2026-10-02
3	P1	1	500	2026-10-03
3	P3	2	50	2026-10-03
4	P2	1	100	2026-10-04
4	P4	1	80	2026-10-04
5	P2	1	100	2026-10-05
5	P3	1	50	2026-10-05
5	P5	1	30	2026-10-05
6	P5	3	30	2026-10-06
7	P1	1	500	2026-10-07
7	P5	1	30	2026-10-07

`product_dimensions`

product_id	category_id	category_name
P1	10	Electronics
P2	20	Books
P3	30	Toys
P4	20	Books
P5	40	Garden

Expected output (ORDER BY times_bought_together DESC, category_pair LIMIT 5)

category_pair	times_bought_together
Books-Electronics	2
Books-Toys	2
Electronics-Toys	2
Books-Garden	1
Electronics-Garden	1

Explanation

- SELECT DISTINCT collapses two products of the same category in one order (order 4 has two Books), so it creates no pair.
- Self-join on order_id pairs categories inside an order; < removes mirror pairs and self-pairs.
- Three pairs tie at 1, so the alphabetical tiebreak decides who is cut by LIMIT 5 (Garden-Toys).

MySQL 8.0+

```
with oc as (
```

```
  select distinct o.order_id, d.category_name
  from orders o
  join product_dimensions d on d.product_id = o.product_id
)
select concat(a.category_name, '-', b.category_name) as category_pair,
       count(*) as times_bought_together
from oc a
join oc b on a.order_id = b.order_id
         and a.category_name < b.category_name     -- each pair once, A-B = B-A
group by a.category_name, b.category_name
order by times_bought_together desc, category_pair
limit 5;
```

Q5 Top reps per region, ties included [Hard]

Using only 2026 sales, compute each rep's total per region. Keep reps whose total is in the top 2 DISTINCT totals of the region (ties share a rank). Exclude regions with fewer than 3 reps having 2026 sales. Return region, rep_name, total_sales, rnk, pct_of_region (share of region total, 2 decimals) and first_sale_date (rep's earliest 2026 sale in that region).

Schema

`sales` (sale_id int, region varchar(20), rep_name varchar(20), sale_date date, amount int)

Sample input

`sales`

sale_id	region	rep_name	sale_date	amount
1	North	Asha	2026-01-05	500
2	North	Asha	2026-02-10	300
3	North	Ravi	2026-01-15	800
4	North	Meena	2026-03-01	400
5	North	Meena	2026-03-20	400
6	North	Kiran	2026-04-02	200
7	South	Tara	2026-01-20	900
8	South	Vik	2026-02-14	600
9	South	Vik	2026-05-01	300
10	South	Neha	2026-03-11	800
11	East	Omar	2026-02-02	700
12	East	Pia	2026-04-18	650
13	West	Zed	2026-01-09	1000
14	West	Zed	2026-06-06	200
15	West	Lia	2025-12-30	5000
16	West	Lia	2026-07-07	1200
17	North	Kiran	2025-11-11	900
18	West	Max	2026-08-08	300

Expected output (ORDER BY region, rnk, first_sale_date, rep_name)

region	rep_name	total_sales	rnk	pct_of_region	first_sale_date
North	Asha	800	1	30.77	2026-01-05
North	Ravi	800	1	30.77	2026-01-15
North	Meena	800	1	30.77	2026-03-01
North	Kiran	200	2	7.69	2026-04-02
South	Tara	900	1	34.62	2026-01-20
South	Vik	900	1	34.62	2026-02-14
South	Neha	800	2	30.77	2026-03-11
West	Zed	1200	1	44.44	2026-01-09
West	Lia	1200	1	44.44	2026-07-07
West	Max	300	2	11.11	2026-08-08

Explanation

- North: Asha, Ravi, Meena tie at 800 (rank 1); Kiran 200 is rank 2 (dense). Kiran's 2025 sale is ignored.
- East has only 2 reps and is excluded. West: Lia's 5000 sale is 2025, so Lia is 1200, tied with Zed.
- Counting reps happens after aggregation to rep level, not on raw rows. Filter the year before aggregating.

MySQL 8.0+

```
with base as (
  select region, rep_name, sum(amount) as total_sales, min(sale_date) as first_sale_date
  from sales
  where sale_date >= '2026-01-01' and sale_date < '2027-01-01'    -- index friendly
  group by region, rep_name
),
region_stats as (
  select region, sum(total_sales) as region_total, count(*) as rep_cnt
  from base group by region having count(*) >= 3
),
ranked as (
  select b.*, r.region_total,
         dense_rank() over (partition by b.region order by b.total_sales desc) as rnk
  from base b join region_stats r on r.region = b.region
)
select region, rep_name, total_sales, rnk,
       round(100.0 * total_sales / region_total, 2) as pct_of_region, first_sale_date
from ranked
where rnk <= 2
order by region, rnk, first_sale_date, rep_name;
```

Note: Spec ambiguity: first_sale_date here is per region+rep. If the interviewer means 'anywhere', compute min(sale_date) grouped by rep_name only. State your assumption.

Q6 Top 2 earners per department, ties broken [Hard]

ACTIVE employees only. Per department rank by salary desc, tie by earliest hire_date then lowest emp_id (unique ranks). Keep top 2. Exclude departments with fewer than 3 active employees. Return dept, emp_name, salary, rn, dept_avg_salary (active employees, 2 decimals) and gap_to_next (salary minus next-ranked active employee's salary, NULL if none).

Schema

`employees` (emp_id int, dept varchar(20), emp_name varchar(20), salary int, hire_date date, status varchar(10))

Sample input

`employees`

emp_id	dept	emp_name	salary	hire_date	status
1	Eng	Anil	150	2020-03-01	ACTIVE
2	Eng	Bela	150	2019-06-15	ACTIVE
3	Eng	Chen	150	2019-06-15	ACTIVE
4	Eng	Dev	120	2021-01-10	ACTIVE
5	Eng	Eli	200	2018-01-01	INACTIVE
6	Sales	Fay	90	2022-02-02	ACTIVE
7	Sales	Gus	95	2021-05-05	ACTIVE
8	Sales	Hari	90	2020-01-01	ACTIVE
9	Sales	Ivy	70	2023-03-03	ACTIVE
10	HR	Jon	80	2020-01-01	ACTIVE
11	HR	Kim	85	2021-01-01	ACTIVE
12	Ops	Lee	100	2019-09-09	ACTIVE
13	Ops	Max	100	2019-09-09	ACTIVE
14	Ops	Nia	60	2022-12-12	ACTIVE
15	Ops	Oz	60	2022-12-12	INACTIVE

Expected output (ORDER BY dept, rn)

dept	emp_name	salary	rn	dept_avg_salary	gap_to_next
Eng	Bela	150	1	142.50	0
Eng	Chen	150	2	142.50	0
Ops	Lee	100	1	86.67	0
Ops	Max	100	2	86.67	40
Sales	Gus	95	1	86.25	5
Sales	Hari	90	2	86.25	0

Explanation

- Eng: Eli is inactive so ignored. Three tie at 150; Bela and Chen share hire date so emp_id decides. Average (450+120)/4 = 142.50.
- HR has only 2 active -> excluded. Ops: Oz inactive leaves 3 active, so Ops stays.
- LEAD is computed BEFORE filtering rn <= 2, so Max (rank 2) still sees Nia (rank 3): gap 40. Never wrap gap in COALESCE(...,0) - NULL means 'no next'.

MySQL 8.0+

```
with base as (
  select emp_id, dept, emp_name, salary,
         row_number() over w as rn,
         lead(salary)  over w as next_salary
  from employees
  where status = 'ACTIVE'
  window w as (partition by dept order by salary desc, hire_date, emp_id)
),
```

```
dep_info as (
  select dept, round(avg(salary), 2) as dept_avg_salary
  from employees where status = 'ACTIVE'
  group by dept having count(*) >= 3
)
select a.dept, a.emp_name, a.salary, a.rn, b.dept_avg_salary,
       a.salary - a.next_salary as gap_to_next          -- NULL when no next employee
from base a join dep_info b on a.dept = b.dept
where a.rn <= 2
order by a.dept, a.rn;
```

Q7 Longest login streak per user [Hard]

A streak is consecutive calendar days with at least one login (several logins on one day count once). For each user find the longest streak; if tied, take the most recent (latest end date). Exclude users whose longest streak is under 2 days. Return user_id, streak_days, streak_start, streak_end.

Schema

`logins` (user_id int, login_date date)

Sample input

`logins`

user_id	login_date
1	2026-10-01
1	2026-10-02
1	2026-10-02
1	2026-10-03
1	2026-10-05
1	2026-10-06
1	2026-10-07
2	2026-10-01
2	2026-10-02
2	2026-10-03
2	2026-10-04
2	2026-10-10
3	2026-10-01
3	2026-10-03
3	2026-10-05
4	2026-10-20
4	2026-10-21
4	2026-10-28
4	2026-10-29
5	2026-10-29
5	2026-10-30
5	2026-10-31

Expected output (ORDER BY streak_days DESC, user_id)

user_id	streak_days	streak_start	streak_end
2	4	2026-10-01	2026-10-04
1	3	2026-10-05	2026-10-07
5	3	2026-10-29	2026-10-31
4	2	2026-10-28	2026-10-29

Explanation

- User 1: duplicate 10-02 counts once; streaks 10-01..03 and 10-05..07 tie at 3, so the later wins. User 3: all isolated -> longest 1 -> excluded.
- Order of steps matters: ranking (rk = 1) is applied before streak_days >= 2.
- Recursive CTE emits every partial length, hence the full_streaks step. It is capped by cte_max_recursion_depth (default 1000).

MySQL 8.0+ (window: login_date minus row_number is constant inside a streak)

```
with base as (
```

```
  select user_id, login_date,
         date_sub(login_date, interval rn day) as grp
  from (
    select user_id, login_date,
           row_number() over (partition by user_id order by login_date) as rn
    from (select distinct user_id, login_date from logins) as d      -- dedupe first
  ) as r
),
streaks as (
  select user_id, min(login_date) as streak_start, max(login_date) as streak_end, count(*) as streak_days
  from base group by user_id, grp
),
ranked as (
  select *, row_number() over (partition by user_id order by streak_days desc, streak_end desc) as rk
  from streaks
)
select user_id, streak_days, streak_start, streak_end
from ranked
where rk = 1 and streak_days >= 2          -- rank first, then filter
order by streak_days desc, user_id;
```

Alternative: recursive CTE

```
with recursive d as (select distinct user_id, login_date from logins),
streaks as (
  select user_id, login_date as streak_start, login_date as streak_end, 1 as streak_days
  from d d1
  where not exists (select 1 from d d2 where d2.user_id = d1.user_id
                    and d2.login_date = date_sub(d1.login_date, interval 1 day))   -- streak starts
  union all
  select s.user_id, s.streak_start, d.login_date, s.streak_days + 1
  from streaks s join d on d.user_id = s.user_id
                       and d.login_date = date_add(s.streak_end, interval 1 day)
),
full_streaks as (
  select user_id, streak_start, max(streak_end) as streak_end, max(streak_days) as streak_days
  from streaks group by user_id, streak_start
),
ranked as (
  select *, row_number() over (partition by user_id order by streak_days desc, streak_end desc) as rk
  from full_streaks
)
select user_id, streak_days, streak_start, streak_end
from ranked where rk = 1 and streak_days >= 2
order by streak_days desc, user_id;
```

Alternative: 'array' style (MySQL has no arrays; JSON array + JSON_TABLE). Not executed in my test environment.

```
set session group_concat_max_len = 1000000;      -- default 1024 truncates long lists
with d as (select distinct user_id, login_date from logins),
arr as (
  select user_id,
         concat('["', replace(group_concat(login_date order by login_date), ',', '","'), '"]') as dates
  from d group by user_id
),
exploded as (
  select a.user_id, str_to_date(j.dt, '%Y-%m-%d') as login_date, j.n
  from arr a, json_table(a.dates, '$[*]' columns (n for ordinality, dt varchar(10) path '$')) j
),
islands as (
  select user_id, min(login_date) as streak_start, max(login_date) as streak_end, count(*) as streak_days
  from exploded group by user_id, date_sub(login_date, interval n day)
),
ranked as (
  select *, row_number() over (partition by user_id order by streak_days desc, streak_end desc) as rk from islands
)
select user_id, streak_days, streak_start, streak_end from ranked
where rk = 1 and streak_days >= 2 order by streak_days desc, user_id;
```

Q8 Second highest distinct salary per department [Medium]

For each department return the second highest DISTINCT salary. If a department has no second distinct salary, return NULL (the department must still appear).

Schema

`employees` (emp_id int, dept varchar(20), emp_name varchar(20), salary int)

Sample input

`employees`

emp_id	dept	emp_name	salary
1	Eng	A	100
2	Eng	B	100
3	Eng	C	90
4	Eng	D	80
5	Sales	E	70
6	Sales	F	70
7	HR	G	60
8	HR	H	50
9	Ops	I	55

Expected output (ORDER BY dept)

dept	second_highest_salary
Eng	90
HR	50
Ops	NULL
Sales	NULL

Explanation

- Eng: distinct salaries 100, 90, 80 -> 90 (the two 100s share rank 1; DENSE_RANK has no gaps).
- Sales: both 70, only one distinct value -> NULL. Ops: single employee -> NULL.
- MAX(CASE WHEN rnk = 2 ...) inside GROUP BY keeps every department and gives NULL when rank 2 does not exist. A WHERE rnk = 2 would drop those departments.

MySQL 8.0+

```
select dept,
       max(case when rnk = 2 then salary end) as second_highest_salary
from (
  select dept, salary,
         dense_rank() over (partition by dept order by salary desc) as rnk
  from employees
) t
group by dept
order by dept;
```

Q9 Duplicate emails: which ids to delete [Medium]

Emails are duplicates when equal ignoring case. Keep the earliest signup (tie: lowest user_id) and list every other row as a delete candidate with the id that is kept. Return delete_id, email, kept_user_id.

Schema

`users` (user_id int, email varchar(50), signup_date date)

Sample input

`users`

user_id	email	signup_date
1	a@x.com	2026-01-01
2	A@x.com	2026-02-01
3	b@x.com	2026-01-05
4	a@x.com	2026-01-01
5	c@x.com	2026-01-10
6	b@x.com	2026-03-01

Expected output (ORDER BY delete_id)

delete_id	email	kept_user_id
2	A@x.com	1
4	a@x.com	1
6	b@x.com	3

Explanation

- Group a@x.com / A@x.com: users 1, 2, 4. User 1 and 4 share a signup date, lowest id (1) wins; 2 and 4 are deletes.
- b@x.com: user 3 kept, user 6 deleted. c@x.com is unique, so it never appears.
- The WINDOW clause defines the window once and reuses it. lower(email) makes the match case-insensitive.

MySQL 8.0+

```
select user_id as delete_id, email, kept_user_id
from (
  select user_id, email,
         row_number() over w as rn,
         first_value(user_id) over w as kept_user_id
  from users
  window w as (partition by lower(email) order by signup_date, user_id)
) t
where rn > 1
order by delete_id;
```

Q10 Month-over-month revenue growth [Medium]

Compute revenue per month, the previous month's revenue and MoM growth percent (2 decimals, NULL for the first month). Return month (YYYY-MM), revenue, prev_revenue, growth_pct.

Schema

`orders` (order_id int, order_date date, amount int)

Sample input

`orders`

order_id	order_date	amount
1	2026-01-05	100
2	2026-01-20	200
3	2026-02-03	200
4	2026-02-25	250
5	2026-03-10	225
6	2026-04-01	120
7	2026-04-28	150

Expected output (ORDER BY month)

month	revenue	prev_revenue	growth_pct
2026-01	300	NULL	NULL
2026-02	450	300	50.00
2026-03	225	450	-50.00
2026-04	270	225	20.00

Explanation

- Jan 300 (no previous month -> NULL). Feb 450: (450-300)/300 = 50.00%. Mar 225: -50.00%. Apr 270: +20.00%.
- LAG(col) looks one row back in the ordered result. It returns NULL for the first row, and NULL propagates through the arithmetic.
- If a month has no orders it is missing from the data, so LAG would compare against the wrong month. Use a calendar table when gaps are possible.

MySQL 8.0+

```
with m as (
  select date_format(order_date, '%Y-%m') as ym, sum(amount) as revenue
  from orders
  group by date_format(order_date, '%Y-%m')
)
select ym as month,
       revenue,
       lag(revenue) over (order by ym) as prev_revenue,
       round(100.0 * (revenue - lag(revenue) over (order by ym))
             / lag(revenue) over (order by ym), 2) as growth_pct
from m
order by ym;
```

Q11 Customers active in every one of the last 3 months [Medium]

Return customers who placed at least one order in each of Aug, Sep and Oct 2026, with their total orders in that period. Return customer_id, active_months, total_orders.

Schema

`orders` (order_id int, customer_id int, order_date date)

Sample input

`orders`

order_id	customer_id	order_date
1	1	2026-08-03
2	1	2026-09-10
3	1	2026-10-05
4	1	2026-10-20
5	2	2026-08-15
6	2	2026-10-01
7	3	2026-09-09
8	3	2026-10-10
9	3	2026-08-30
10	4	2026-07-30
11	4	2026-08-12
12	4	2026-09-12
13	5	2026-10-02

Expected output (ORDER BY customer_id)

customer_id	active_months	total_orders
1	3	4
3	3	3

Explanation

- Customer 1 has Aug, Sep, Oct (Oct twice) -> 3 months, 4 orders. Customer 3 has all three months.
- Customer 2 misses Sep. Customer 4's July order is outside the window, so only Aug and Sep count. Customer 5 has one month.
- Filter the date range in WHERE (cheap, index friendly) then count distinct months in HAVING.

MySQL 8.0+

```
select customer_id,
       count(distinct date_format(order_date, '%Y-%m')) as active_months,
       count(*) as total_orders
from orders
where order_date >= '2026-08-01' and order_date < '2026-11-01'
group by customer_id
having count(distinct date_format(order_date, '%Y-%m')) = 3
order by customer_id;
```

Q12 Employees earning more than their manager [Medium]

Return employees who earn more than their direct manager, with the manager's name and both salaries and the difference. Return emp_name, manager_name, salary, manager_salary, diff.

Schema

`emp` (emp_id int, emp_name varchar(20), salary int, manager_id int)

Sample input

`emp`

emp_id	emp_name	salary	manager_id
1	Ceo	300	NULL
2	Ann	200	1
3	Bob	250	2
4	Cal	180	2
5	Dee	320	1
6	Eve	150	3

Expected output (ORDER BY diff DESC, emp_name)

emp_name	manager_name	salary	manager_salary	diff
Bob	Ann	250	200	50
Dee	Ceo	320	300	20

Explanation

- Bob (250) > Ann (200): diff 50. Dee (320) > Ceo (300): diff 20.
- Ceo has no manager (NULL manager_id) so the inner join drops that row. Use LEFT JOIN only if you must keep top-level people.
- Self-join pattern: alias the same table twice (e = employee, m = manager) and join employee.manager_id = manager.emp_id.

MySQL 8.0+

```
select e.emp_name,
       m.emp_name as manager_name,
       e.salary,
       m.salary   as manager_salary,
       e.salary - m.salary as diff
from emp e
join emp m on m.emp_id = e.manager_id
where e.salary > m.salary
order by diff desc, e.emp_name;
```

Q13 Pivot: monthly sales by category [Medium]

Turn rows into columns: one row per month, one column per category (Books, Toys, Food) with 0 where there were no sales, plus a total. Return month, Books, Toys, Food, total.

Schema

`sales` (sale_date date, category varchar(20), amount int)

Sample input

`sales`

sale_date	category	amount
2026-01-05	Books	100
2026-01-18	Books	50
2026-01-20	Toys	70
2026-02-02	Food	40
2026-02-14	Books	60
2026-02-20	Toys	30
2026-03-03	Food	90
2026-03-09	Food	10

Expected output (ORDER BY month)

month	Books	Toys	Food	total
2026-01	150	70	0	220
2026-02	60	30	40	130
2026-03	0	0	100	100

Explanation

- Each SUM(CASE ...) is a manual PIVOT column. ELSE 0 stops NULL for missing combinations.
- Jan: Books 150, Toys 70, Food 0, total 220. Mar has only Food (100).
- MySQL has no PIVOT keyword. The categories must be listed in the query (dynamic pivots need prepared statements).

MySQL 8.0+

```
select date_format(sale_date, '%Y-%m') as month,
       sum(case when category = 'Books' then amount else 0 end) as Books,
       sum(case when category = 'Toys'  then amount else 0 end) as Toys,
       sum(case when category = 'Food'  then amount else 0 end) as Food,
       sum(amount) as total
from sales
group by date_format(sale_date, '%Y-%m')
order by month;
```

Q14 Median salary per department [Hard]

MySQL has no MEDIAN function. Return the median salary per department. For an even number of employees the median is the average of the two middle values. Return dept and median_salary (2 decimals).

Schema

`employees` (emp_id int, dept varchar(20), salary int)

Sample input

`employees`

emp_id	dept	salary
1	Eng	100
2	Eng	200
3	Eng	300
4	Eng	400
5	Sales	50
6	Sales	70
7	Sales	90
8	HR	10
9	Ops	20
10	Ops	20
11	Ops	40
12	Ops	60
13	Ops	80

Expected output (ORDER BY dept)

dept	median_salary
Eng	250.00
HR	10.00
Ops	40.00
Sales	70.00

Explanation

- Odd count n: both floor expressions give the middle position, so one row is averaged. Even count: they give the two middle positions.
- Eng (4 rows): positions 2 and 3 -> (200+300)/2 = 250. Sales (3): position 2 -> 70. HR (1): 10. Ops (5): position 3 -> 40.
- Use floor((cnt+1)/2): in MySQL `/` is decimal division, so (cnt+1)/2 alone would be 2.5 for cnt = 4.

MySQL 8.0+

```
with r as (
  select dept, salary,
         row_number() over (partition by dept order by salary, emp_id) as rn,
         count(*)     over (partition by dept) as cnt
  from employees
)
select dept, round(avg(salary), 2) as median_salary
from r
where rn in (floor((cnt + 1) / 2), floor((cnt + 2) / 2))
group by dept
order by dept;
```

Q15 Products never sold, and dead categories [Medium]

List products that never appear in any order. For each, flag whether the whole category is unsold (Y) or the category has other products that sold (N). Return product_name, category, category_fully_unsold.

Schema

`products` (product_id int, name varchar(20), category varchar(20))
`order_items` (order_id int, product_id int, qty int)

Sample input

`products`

product_id	name	category
1	Pen	Stationery
2	Pencil	Stationery
3	Mug	Kitchen
4	Plate	Kitchen
5	Lamp	Decor
6	Vase	Decor

`order_items`

order_id	product_id	qty
1	1	2
1	3	1
2	1	1
3	3	4

Expected output (ORDER BY category, product_name)

product_name	category	category_fully_unsold
Lamp	Decor	Y
Vase	Decor	Y
Plate	Kitchen	N
Pencil	Stationery	N

Explanation

- Pen and Mug sold. Pencil (Stationery) and Plate (Kitchen) never sold but their categories each have a seller -> N.
- Lamp and Vase are both unsold and nothing else in Decor sold -> Y.
- LEFT JOIN ... WHERE right.key IS NULL is the anti-join. NOT EXISTS answers the category-level question.

MySQL 8.0+

```
select p.name as product_name, p.category,
       case when not exists (select 1
                             from products p2
                             join order_items oi2 on oi2.product_id = p2.product_id
                             where p2.category = p.category) then 'Y' else 'N' end as category_fully_unsold
from products p
left join order_items oi on oi.product_id = p.product_id
where oi.product_id is null                       -- anti-join: never sold
order by p.category, p.name;
```

Q16 First day cumulative sales reach 1000 [Hard]

For each region find the first date on which cumulative sales (ordered by date) reach or exceed 1000, and the cumulative value on that date. Regions that never reach 1000 are excluded. Return region, crossing_date, cumulative_sales.

Schema

`daily_sales` (region varchar(20), sale_date date, amount int)

Sample input

`daily_sales`

region	sale_date	amount
North	2026-01-01	400
North	2026-01-02	300
North	2026-01-03	500
South	2026-01-01	600
South	2026-01-02	300
South	2026-01-03	100
West	2026-01-01	200
West	2026-01-02	300
East	2026-01-01	1500
East	2026-01-02	100

Expected output (ORDER BY region)

region	crossing_date	cumulative_sales
East	2026-01-01	1500
North	2026-01-03	1200
South	2026-01-03	1000

Explanation

- North: 400, 700, 1200 -> crossed on Jan 3. South: exactly 1000 on Jan 3 (>= includes equality). East crosses on day one.
- West tops out at 500 and never appears.
- Running totals are cumulative, so once cum >= 1000 every later row also passes. Taking rn = 1 among them gives the first crossing.

MySQL 8.0+

```
with c as (
  select region, sale_date,
         sum(amount) over (partition by region order by sale_date) as cum
  from daily_sales
)
select region, sale_date as crossing_date, cum as cumulative_sales
from (
  select c.*, row_number() over (partition by region order by sale_date) as rn
  from c
  where cum >= 1000                    -- keep only rows after crossing, then take the first
) t
where rn = 1
order by region;
```

Q17 Sessionization with a 30-minute gap [Hard]

A new session starts when the gap since the user's previous event is MORE than 30 minutes (exactly 30 is the same session). Return per user and session: user_id, session_no, session_start, session_end, events, duration_min.

Schema

`events` (user_id int, event_time datetime)

Sample input

`events`

user_id	event_time
1	2026-10-30 10:00:00
1	2026-10-30 10:10:00
1	2026-10-30 10:25:00
1	2026-10-30 11:05:00
1	2026-10-30 11:20:00
2	2026-10-30 09:00:00
2	2026-10-30 09:30:00
2	2026-10-30 10:01:00

Expected output (ORDER BY user_id, session_no)

user_id	session_no	session_start	session_end	events	duration_min
1	1	2026-10-30 10:00:00	2026-10-30 10:25:00	3	25
1	2	2026-10-30 11:05:00	2026-10-30 11:20:00	2	15
2	1	2026-10-30 09:00:00	2026-10-30 09:30:00	2	30
2	2	2026-10-30 10:01:00	2026-10-30 10:01:00	1	0

Explanation

- Flag rows that start a session (first event or gap > 1800 seconds), then a running SUM of the flag numbers the sessions.
- User 1: 10:00, 10:10, 10:25 form session 1; the 40 minute gap starts session 2. User 2: the gap of exactly 30 minutes stays in session 1, 31 minutes starts session 2.
- Compare in seconds: TIMESTAMPDIFF(MINUTE, ...) truncates, so a 30m30s gap would wrongly look like 30 minutes.

MySQL 8.0+

```
with g as (
  select user_id, event_time,
         case when lag(event_time) over w is null
                or timestampdiff(second, lag(event_time) over w, event_time) > 1800
              then 1 else 0 end as new_sess
  from events
  window w as (partition by user_id order by event_time)
),
s as (
  select user_id, event_time,
         sum(new_sess) over (partition by user_id order by event_time) as session_no
  from g
)
select user_id, session_no,
       min(event_time) as session_start, max(event_time) as session_end,
       count(*) as events,
       timestampdiff(minute, min(event_time), max(event_time)) as duration_min
from s
group by user_id, session_no
order by user_id, session_no;
```

Q18 Monthly cohort retention [Hard]

Group users by signup month (cohort). For each cohort return its size and the percent of users active in month 0, 1 and 2 after signup. Return cohort_month, cohort_size, m0, m1, m2 (percent, 2 decimals).

Schema

`users` (user_id int, signup_date date)
`activity` (user_id int, activity_date date)

Sample input

`users`

user_id	signup_date
1	2026-01-04
2	2026-01-10
3	2026-01-15
4	2026-01-28
5	2026-02-03
6	2026-02-20

`activity`

user_id	activity_date
1	2026-01-05
1	2026-02-07
1	2026-03-02
2	2026-01-11
2	2026-02-12
3	2026-01-16
4	2026-01-29
4	2026-03-15
5	2026-02-04
5	2026-03-05
5	2026-04-09
6	2026-02-21

Expected output (ORDER BY cohort_month)

cohort_month	cohort_size	m0	m1	m2
2026-01	4	100.00	50.00	50.00
2026-02	2	100.00	50.00	50.00

Explanation

- Jan cohort = users 1-4. Month 1 (Feb): users 1 and 2 -> 50%. Month 2 (Mar): users 1 and 4 -> 50%.
- Feb cohort = users 5, 6. Month 1 (Mar): user 5 -> 50%. Month 2 (Apr): user 5 -> 50%.
- The denominator comes from the cohort-size CTE, not from joined activity rows, otherwise users with no later activity would vanish from the denominator.

MySQL 8.0+

```
with c as (
  select user_id, date_format(signup_date, '%Y-%m-01') as cohort from users
),
sz as (select cohort, count(*) as n from c group by cohort),
a as (select distinct user_id, date_format(activity_date, '%Y-%m-01') as act_month from activity),
j as (
  select c.cohort, c.user_id,
         (year(a.act_month) - year(c.cohort)) * 12 + month(a.act_month) - month(c.cohort) as m
  from c join a on a.user_id = c.user_id
```

```
)
select substr(j.cohort, 1, 7) as cohort_month, sz.n as cohort_size,
       round(100.0 * count(distinct case when m = 0 then j.user_id end) / sz.n, 2) as m0,
       round(100.0 * count(distinct case when m = 1 then j.user_id end) / sz.n, 2) as m1,
       round(100.0 * count(distinct case when m = 2 then j.user_id end) / sz.n, 2) as m2
from j join sz on sz.cohort = j.cohort
group by j.cohort, sz.n
order by j.cohort;
```

Note: In real data, months that have not happened yet should show NULL, not 0%. Add a condition on the max observed month.

Q19 Overlapping room bookings [Hard]

A booking occupies [check_in, check_out), so a checkout on the same day as another check-in is NOT an overlap. Return every pair of overlapping bookings in the same room once. Return room_id, booking_a, booking_b.

Schema

`bookings` (booking_id int, room_id int, check_in date, check_out date)

Sample input

`bookings`

booking_id	room_id	check_in	check_out
1	1	2026-10-01	2026-10-05
2	1	2026-10-04	2026-10-08
3	1	2026-10-08	2026-10-10
4	2	2026-10-01	2026-10-03
5	2	2026-10-05	2026-10-06
6	3	2026-10-01	2026-10-10
7	3	2026-10-02	2026-10-03
8	3	2026-10-09	2026-10-12

Expected output (ORDER BY room_id, booking_a, booking_b)

room_id	booking_a	booking_b
1	1	2
3	6	7
3	6	8

Explanation

- Two intervals overlap exactly when each one starts before the other ends.
- Room 1: 1 and 2 overlap (Oct 4-5). Booking 3 starts the day booking 2 ends -> no overlap. Room 3: 6 contains 7 and overlaps 8.
- Use < not <= for half-open intervals. If check_out is inclusive use <=.

MySQL 8.0+

```
select a.room_id, a.booking_id as booking_a, b.booking_id as booking_b
from bookings a
join bookings b on a.room_id = b.room_id
               and a.booking_id < b.booking_id          -- each pair once
               and a.check_in  < b.check_out
               and b.check_in  < a.check_out            -- classic overlap test
order by a.room_id, booking_a, booking_b;
```

Q20 Merge overlapping date ranges [Hard]

Per user, merge overlapping or touching ranges (a range whose start is <= the merged range's current end is merged). Ranges that leave a gap of at least one day stay separate. Return user_id, merged_start, merged_end.

Schema

`intervals` (user_id int, start_date date, end_date date)

Sample input

`intervals`

user_id	start_date	end_date
1	2026-01-01	2026-01-05
1	2026-01-03	2026-01-08
1	2026-01-08	2026-01-10
1	2026-01-15	2026-01-16
2	2026-02-01	2026-02-10
2	2026-02-02	2026-02-03
2	2026-02-11	2026-02-12
3	2026-03-01	2026-03-02

Expected output (ORDER BY user_id, merged_start)

user_id	merged_start	merged_end
1	2026-01-01	2026-01-10
1	2026-01-15	2026-01-16
2	2026-02-01	2026-02-10
2	2026-02-11	2026-02-12
3	2026-03-01	2026-03-02

Explanation

- prev_max_end is the furthest end seen among earlier ranges. If the next start is beyond it, a new block begins; a running SUM of those flags is the block id.
- User 1: Jan 1-5, 3-8, 8-10 chain into Jan 1-10 (touching on Jan 8 merges). Jan 15-16 is separate.
- User 2: Feb 2-3 sits inside Feb 1-10. Feb 11 starts after Feb 10, so it stays separate.
- A plain LAG(end_date) is not enough: one long range can swallow several later short ones, so the running MAX is required.

MySQL 8.0+

```
with f as (
  select user_id, start_date, end_date,
         max(end_date) over (partition by user_id order by start_date, end_date
                             rows between unbounded preceding and 1 preceding) as prev_max_end
  from intervals
),
g as (
  select *, sum(case when prev_max_end is null or start_date > prev_max_end then 1 else 0 end)
              over (partition by user_id order by start_date, end_date) as grp
  from f
)
select user_id, min(start_date) as merged_start, max(end_date) as merged_end
from g
group by user_id, grp
order by user_id, merged_start;
```

Q21 All reports under a manager (recursive CTE) [Hard]

Return every direct and indirect report of employee 1 (Asha), with their depth below Asha (direct reports = 1) and the full reporting path. Return emp_name, level, path.

Schema

`employees` (emp_id int, emp_name varchar(20), manager_id int)

Sample input

`employees`

emp_id	emp_name	manager_id
1	Asha	NULL
2	Ben	1
3	Cara	1
4	Dev	2
5	Esha	2
6	Farid	4
7	Gita	3
8	Hari	NULL
9	Ivan	8

Expected output (ORDER BY path)

emp_name	level	path
Ben	1	Ben
Dev	2	Ben > Dev
Farid	3	Ben > Dev > Farid
Esha	2	Ben > Esha
Cara	1	Cara
Gita	2	Cara > Gita

Explanation

- The anchor row set is Asha's direct reports. Each recursion step joins employees whose manager is a row found in the previous step.
- Hari and Ivan sit in a different tree and never appear. Recursion stops when a step returns no new rows.
- CAST(... AS CHAR(200)) fixes the column width of the recursive path; without it MySQL may truncate or error on longer paths.

MySQL 8.0+

```
with recursive t as (
  select emp_id, emp_name, 1 as lvl, cast(emp_name as char(200)) as path
  from employees
  where manager_id = 1                                  -- anchor: direct reports
  union all
  select e.emp_id, e.emp_name, t.lvl + 1, concat(t.path, ' > ', e.emp_name)
  from employees e
  join t on e.manager_id = t.emp_id                     -- step: reports of the previous level
)
select emp_name, lvl as level, path
from t
order by path;
```

Q22 People you may know (mutual friends) [Hard]

`friendships` stores each friendship once (either direction). For every user suggest the NON-friend with the most mutual friends; ties go to the smaller user id. Only users with at least one mutual friend get a suggestion. Return user_id, suggested_friend, mutual_friends.

Schema

`friendships` (user_id int, friend_id int)

Sample input

`friendships`

user_id	friend_id
1	2
1	3
2	4
3	4
3	5
4	6
5	6
6	7

Expected output (ORDER BY user_id)

user_id	suggested_friend	mutual_friends
1	4	2
2	3	2
3	2	2
4	1	2
5	4	2
6	3	2
7	4	1

Explanation

- UNION of both directions turns the one-row-per-pair table into an adjacency list.
- User 3 ties between candidates 2 and 6 (2 mutual each) -> smaller id 2. User 7 only knows 6, so candidates 4 and 5 tie at 1 -> 4.
- Joining e a to e b on a.v = b.u walks two hops; counting rows per (user, candidate) counts mutual friends.

MySQL 8.0+

```
with e as (                                  -- make the graph symmetric
  select user_id as u, friend_id as v from friendships
  union
  select friend_id, user_id from friendships
),
m as (
  select a.u as user_id, b.v as suggested, count(*) as mutual
  from e a
  join e b on a.v = b.u                      -- a.u - a.v - b.v : a.v is the mutual friend
  where a.u <> b.v
    and not exists (select 1 from e x where x.u = a.u and x.v = b.v)   -- not already friends
  group by a.u, b.v
),
r as (
  select *, row_number() over (partition by user_id order by mutual desc, suggested) as rn from m
)
select user_id, suggested as suggested_friend, mutual as mutual_friends
from r
where rn = 1
order by user_id;
```

Q23 Rolling 7-calendar-day average with missing days [Hard]

Days with no sales have no row. For each sale_date compute the 7-calendar-day rolling average revenue: sum of revenue over that day and the 6 days before, divided by 7 (missing days count as 0). Return sale_date, revenue, rolling_7d_avg.

Schema

`daily` (sale_date date, revenue int)

Sample input

`daily`

sale_date	revenue
2026-10-01	70
2026-10-02	70
2026-10-04	140
2026-10-08	35
2026-10-09	105
2026-10-12	210

Expected output (ORDER BY sale_date)

sale_date	revenue	rolling_7d_avg
2026-10-01	70	10.00
2026-10-02	70	20.00
2026-10-04	140	40.00
2026-10-08	35	35.00
2026-10-09	105	40.00
2026-10-12	210	50.00

Explanation

- A ROWS frame would take the last 7 ROWS, which is wrong when days are missing. A date-based frame (RANGE with INTERVAL, or the correlated subquery) uses calendar days.
- Oct 8: window Oct 2-8 = 70 + 140 + 35 = 245 -> 35.00. Oct 12: window Oct 6-12 = 35 + 105 + 210 = 350 -> 50.00.
- Divide by 7.0 (constant) because missing days count as zero; AVG() would divide by the number of rows present.

MySQL 8.0+

```
select d.sale_date, d.revenue,
       round((select sum(d2.revenue) from daily d2
              where d2.sale_date between date_sub(d.sale_date, interval 6 day) and d.sale_date) / 7.0, 2)
         as rolling_7d_avg
from daily d
order by d.sale_date;
```

Alternative: RANGE window (MySQL 8.0). Not executed in my test environment.

```
select sale_date, revenue,
       round(sum(revenue) over (order by sale_date
                                range between interval 6 day preceding and current row) / 7.0, 2)
         as rolling_7d_avg
from daily
order by sale_date;
```

Q24 Top-selling product per category with share [Medium]

Return the best-selling product(s) in each category by revenue (all ties) and each one's percent share of the category's revenue (2 decimals). Return category, product_name, revenue, pct_of_category.

Schema

`product_sales` (product_id int, product_name varchar(20), category varchar(20), revenue int)

Sample input

`product_sales`

product_id	product_name	category	revenue
1	B1	Books	500
2	B2	Books	500
3	B3	Books	200
4	T1	Toys	300
5	T2	Toys	100
6	F1	Food	250

Expected output (ORDER BY category, product_name)

category	product_name	revenue	pct_of_category
Books	B1	500	41.67
Books	B2	500	41.67
Food	F1	250	100.00
Toys	T1	300	75.00

Explanation

- Books: B1 and B2 tie at 500 (rank 1 for both); category total 1200 -> 41.67% each. Toys: T1 300 of 400 -> 75.00%. Food: 100.00%.
- The share is computed in the same inner query, before the rk = 1 filter, so the denominator is the full category total.
- RANK keeps ties. Swap to ROW_NUMBER with a tiebreak if exactly one product per category is needed.

MySQL 8.0+

```
select category, product_name, revenue, pct_of_category
from (
  select category, product_name, revenue,
         rank() over (partition by category order by revenue desc) as rk,
         round(100.0 * revenue / sum(revenue) over (partition by category), 2) as pct_of_category
  from product_sales
) t
where rk = 1
order by category, product_name;
```

Q25 Users who flipped status at least twice [Medium]

A subscription log records ACTIVE/PAUSED events. A flip is an event whose status differs from the same user's previous event. Return users with at least 2 flips and the flip count. Return user_id, flips.

Schema

`subscription_log` (user_id int, event_date date, status varchar(10))

Sample input

`subscription_log`

user_id	event_date	status
1	2026-01-01	ACTIVE
1	2026-02-01	PAUSED
1	2026-03-01	ACTIVE
2	2026-01-01	ACTIVE
2	2026-02-01	ACTIVE
2	2026-03-01	PAUSED
3	2026-01-01	ACTIVE
3	2026-02-01	PAUSED
3	2026-03-01	ACTIVE
3	2026-04-01	PAUSED
4	2026-01-01	ACTIVE
5	2026-01-01	PAUSED
5	2026-02-01	ACTIVE
5	2026-03-01	ACTIVE
5	2026-04-01	PAUSED

Expected output (ORDER BY flips DESC, user_id)

user_id	flips
3	3
1	2
5	2

Explanation

- User 3: A, P, A, P = 3 flips. User 1: A, P, A = 2. User 5: P, A, A, P = 2 (the repeated ACTIVE is not a flip).
- User 2 has 1 flip; user 4 has none, so neither is returned.
- The first event has prev_status NULL; `status <> NULL` is NULL, so guard with `prev_status is not null` (or rely on the CASE falling to ELSE 0).

MySQL 8.0+

```
select user_id, sum(case when prev_status is not null and status <> prev_status then 1 else 0 end) as flips
from (
  select user_id, status,
         lag(status) over (partition by user_id order by event_date) as prev_status
  from subscription_log
) t
group by user_id
having flips >= 2
order by flips desc, user_id;
```

MySQL quick reference

Task	MySQL 8.0+ syntax	Gotcha
First / last char	LEFT(c,1) / RIGHT(c,1); SUBSTR(c,start,len)	1-indexed; len must be positive
Concatenate	CONCAT(a,'-',b); CONCAT_WS('-',a,b)	|| is OR, CONCAT gives NULL if any arg NULL
Conditional count	COUNT(CASE WHEN x THEN 1 END); SUM(x = 'A')	SUM over no rows is NULL; COUNT is 0
Date shift	DATE_ADD(d, INTERVAL 7 DAY); DATE_SUB(...)	Month math clamps to month end
Date diff	DATEDIFF(later, earlier); TIMESTAMPDIFF(SECOND, a, b)	TIMESTAMPDIFF(unit, earlier, later)
Truncate to month	DATE_FORMAT(d,'%Y-%m-01')	No DATE_TRUNC; returns a string
Week start (Mon)	DATE_SUB(d, INTERVAL WEEKDAY(d) DAY)	WEEKDAY: Mon=0; DAYOFWEEK: Sun=1
Running total	SUM(x) OVER (PARTITION BY g ORDER BY d)	Add tiebreaker; default frame is RANGE
Running avg	AVG(x) OVER (... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)	ROWS treats ties as separate rows
Top N per group	ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC) in a CTE, WHERE rn <= N	No QUALIFY: filter in outer query
Ties	RANK (gaps), DENSE_RANK (no gaps), ROW_NUMBER (unique)	RANK is a reserved word: alias it rnk
Previous / next row	LAG(x) OVER (...), LEAD(x) OVER (...)	NULL at the edges
Named window	... OVER w ... WINDOW w AS (PARTITION BY a ORDER BY b)	WINDOW comes after HAVING
Gaps and islands	date_sub(d, INTERVAL ROW_NUMBER() ... DAY) as grp, then GROUP BY grp	DISTINCT the dates first
Recursive CTE	WITH RECURSIVE t AS (anchor UNION ALL step) SELECT ...	cte_max_recursion_depth = 1000
Median	ROW_NUMBER + COUNT window, rn IN (FLOOR((n+1)/2), FLOOR((n+2)/2))	No MEDIAN function
Limit	LIMIT n or LIMIT offset, n	No FETCH FIRST
Pivot	SUM(CASE WHEN col='X' THEN v ELSE 0 END) per column	No PIVOT keyword
Division	a / b is decimal; a DIV b is integer	Use 100.0 * a / b for percents